

Document number	P2485R0
Date	2021-11-01
Audience	LEWG
Reply-to	Jonathan Wakely <cxx@kayari.org>

Do not add `value_exists` and `value_or` to C++23

- - [Abstract](#)
 - [Discussion](#)
 - [Background](#)
 - [Proposed change](#)

Abstract

Drop the `value_exists` and `value_or` helpers from [P1841](#) and only adopt the numeric traits that are the core of the proposal.

Discussion

[\[P1841R1\]](#) (Wording for Individually Specializable Numeric Traits) proposes to add:

```
template <template<class> class Trait, class T>
constexpr bool value_exists = requires { Trait<T>::value; };
```

(The paper actually says *see below* and doesn't provide the definition, but it probably should provide it as above.)

During review of P1841R1 LWG wanted to know why `value_exists` is used as `value_exists<foo, T>` rather than `value_exists<foo<T>>`. Why is the proposed design that the `value_exists` template "applies" the non-type template argument to the second template argument? The following seems simpler to understand for non-expert users:

```
template <class T>
constexpr bool value_exists = requires { T::value };;
```

With this alternative API you would say `value_exists<signaling_NaN<float>>` not `value_exists<signaling_NaN, float>`.

LWG also noted that the proposed API for `value_or` uses the argument type as the return type, so that `value_or<finite_min, double>(1)` is rounded to `int`. This is surprising.

On the LEWG reflector I asked whether there is any rationale for the proposed design of `value_exists`, and whether we need this utility at all. There was support for dropping `value_exists` and `value_or` from the proposal.

Background

[\[P0437R1\]](#) (Numeric Traits for the Standard Library) introduced the `value_exists` utility, but didn't justify the API choice. It also pointed out that the adoption of [\[P0266R1\]](#) (Lifting Restrictions on `requires`-Expressions) would make it unnecessary. P0266R1 was adopted at the Issaquah 2016 meeting.

In San Diego the LEWG discussion of P0437R1 included:

"value_exists belongs in compile-time programming support"

"has_value_or — if not exists, use arg. usefully coupled, but separable."

Want value_exists?

0 5 6 0 1

That is quite weak support. There is no record in the minutes (or that I recall from that session) about the proposed API.

Proposed change

I propose that we drop the `value_exists` and `value_or` helpers from P1841 and only adopt the actual numeric traits. If we want the metaprogramming helpers, they can be added separately at a later date.